Trabalho Pratico 02 - Patrulha Antidesmatamento (UDP + Threads em C)

Redes de Computadores — 2025/2

Aluno: João Vaz Matrícula: 2023099166

2 de dezembro de 2025

1 Introdução

O presente trabalho implementa um sistema distribuído composto por dois binários independentes: um **servidor** e um **cliente**, que se comunicam por meio do protocolo UDP. O objetivo geral é simular o monitoramento ambiental de regiões da Amazônia Legal, detectando possíveis situações de alerta e despachando equipes de drones para atuação nesses pontos.

O sistema integra três pilares fundamentais:

- **Comunicação em rede via UDP**: responsável pela transmissão periódica de telemetria, notificações de despacho de drones e conclusões de missão.
- **Concorrência via threads**: utilizada no cliente para simular sensores, comunicação assíncrona e execução de missões.
- **Estruturas de dados e algoritmos**: representadas principalmente pelo grafo da Amazônia Legal e pelo uso do algoritmo de Dijkstra para seleção da equipe de drones mais próxima de um alerta.

Cada mensagem trafegada entre cliente e servidor segue o formato *header + payload*, permitindo que ambos os programas possam identificar o tipo e o tamanho de cada datagrama recebido, e assim processá-lo de acordo com seu papel no sistema.

O comportamento global pode ser resumido da seguinte forma:

1. O cliente simula leituras de sensores em todas as cidades monitoradas, atualizando periodicamente o status de cada uma (normal ou alerta).
2. A cada 30 segundos, a thread de telemetria envia ao servidor uma mensagem `MSG_TELEMETRIA` contendo o estado completo das cidades. O cliente aguarda um `MSG_ACK`; caso não o receba no tempo limite, tenta um número limitado de retransmissões.
3. Ao receber telemetria, o servidor identifica quais cidades estão em alerta, registra cada evento e executa o algoritmo de Dijkstra para determinar qual capital (e portanto, qual equipe de drones) está mais próxima para atuar.
4. O servidor envia ao cliente uma mensagem `MSG_EQUIPE_DRONE` indicando qual equipe deve ser enviada à cidade em alerta. O cliente responde com um `MSG_ACK`.

5. O cliente simula a atuação da equipe de drones durante um intervalo de até 30 segundos. Ao final, envia ao servidor uma mensagem `MSG_CONCLUSAO` contendo o identificador da cidade atendida e da equipe atuante.

6. O servidor libera a equipe para novas missões e envia um `MSG_ACK` confirmando a conclusão.

Cada etapa descrita acima foi cuidadosamente implementada de forma a garantir:

- a correta interpretação de todos os tipos de mensagens definidas no protocolo;
- tratamento explícito de erros, tais como timeouts, mensagens inválidas ou desconhecidas;
- sincronização consistente entre threads no cliente;
- integridade dos dados no servidor, incluindo controle de cidades, equipes e registros de alerta;
- operação transparente tanto em IPv4 quanto em IPv6, conforme definido na especificação.

Esta documentação tem como objetivo detalhar cada uma dessas partes, explicando não apenas *o que* foi implementado, mas também *por que* cada decisão foi tomada. Ao longo das próximas seções descrevemos o protocolo utilizado, as estruturas de dados, o funcionamento de cada módulo (servidor e cliente) e o fluxo completo de mensagens de ponta a ponta.

2 Protocolo de Comunicação UDP

A comunicação entre servidor e cliente é realizada exclusivamente por meio do protocolo UDP. Todas as mensagens seguem o formato **binário fixo** definido no enunciado, consistindo de um cabeçalho (*header*) seguido de um bloco de dados (*payload*). O servidor e o cliente tratam cada datagrama recebido de forma independente, sem qualquer garantia de entrega, duplicação ou ordenação, o que exige tratamento explícito de erros e retransmissões.

A seguir descrevemos a estrutura do cabeçalho, os tipos de mensagens suportados e o comportamento esperado de cada lado da comunicação ao processá-las.

2.1 Estrutura do Cabeçalho

O cabeçalho (`header_t`) é composto por:

- **tipo** (inteiro de 32 bits): indica o tipo da mensagem;
- **tamanho** (inteiro de 32 bits): tamanho em bytes do payload.

Este cabeçalho permite que qualquer lado identifique e valide a mensagem recebida, mesmo quando o datagrama chega truncado, incompleto ou com tipo desconhecido. O programa verifica:

- se o tamanho do pacote é menor que o cabeçalho, ele é descartado;
- se o campo `tamanho` é menor que o esperado para aquele tipo de mensagem, a mensagem é tratada como inválida.

2.2 Tipos de Mensagem

O sistema utiliza quatro tipos principais de mensagens, definidos como inteiros:

- `MSG_TELEMETRIA` (1)
- `MSG_EQUIPE_DRONE` (2)
- `MSG_CONCLUSAO` (3)
- `MSG_ACK` (0)

Cada um deles possui um payload próprio, com estrutura definida no arquivo `protocol.h`. A seguir, detalhamos o papel de cada tipo e o comportamento específico no cliente e no servidor.

2.3 `MSG_TELEMETRIA` (Cliente → Servidor)

A mensagem de telemetria contém o estado de todas as cidades monitoradas, incluindo o identificador da cidade e o status (`0 = normal`, `1 = alerta`). O payload é composto por:

- número total de cidades monitoradas;
- um vetor de pares *(id, status)*.

Ao **receber** uma mensagem de telemetria:

No servidor:

1. Valida-se o tamanho do payload e o número de cidades.
2. Cada par *(id, status)* é verificado; ids inválidos são ignorados.
3. Para cada cidade em estado de alerta:
    - verifica-se se já existe um alerta registrado;
    - se não houver, cria-se um novo registro de alerta;
    - executa-se Dijkstra com origem na cidade em alerta;
    - escolhe-se a capital mais próxima com equipe livre;
    - envia-se `MSG_EQUIPE_DRONE`.
4. O servidor envia ao cliente um **`MSG_ACK`** confirmando o recebimento.

No cliente:

- Esta mensagem nunca deve ser recebida; caso apareça, o cliente registra o evento e a descarta.

Erros e comportamentos inesperados:

- se o tamanho do payload for menor que o esperado, a telemetria é descartada;
- se o servidor não enviar `ACK`, a thread de telemetria do cliente realiza até três tentativas de retransmissão;
- se todas as tentativas falharem, o cliente aguarda 30 segundos e reinicia o ciclo.

2.4 `MSG_EQUIPE_DRONE` (Servidor → Cliente)

Esta mensagem é enviada pelo servidor quando uma cidade entra em estado de alerta e uma equipe de drones precisa ser despachada. O payload contém:

- `id_cidade`: cidade em alerta;
- `id_equipe`: equipe selecionada (sempre uma capital).

No cliente: Ao receber `MSG_EQUIPE_DRONE`, o cliente:

1. Lê o par *(id_cidade, id_equipe)*.
2. Sinaliza a thread de simulação de drones para iniciar a missão.
3. Envia de volta ao servidor um `MSG_ACK` com status `ACK_EQ_EQUIPE`.

No servidor:

- Esta mensagem não deve ser recebida; se ocorrer, o servidor apenas loga e ignora.

Erros e comportamentos inesperados:

- se o payload tiver tamanho incorreto, a mensagem é descartada;
- se o cliente estiver em missão e receber outra ordem, a nova substitui a anterior.

2.5 `MSG_CONCLUSAO` (Cliente → Servidor)

Enviada quando o cliente finaliza a simulação da missão. O payload contém:

- identificador da cidade atendida;
- identificador da equipe responsável.

No servidor: Ao receber `MSG_CONCLUSAO`, o servidor:

1. Marca a equipe como livre novamente.
2. Finaliza o alerta associado àquela cidade.
3. Envia um `MSG_ACK` com o status `ACK_CONCLUSAO`.

No cliente:

- O cliente inicia um *timeout* de 5 segundos aguardando o ACK;
- se não receber, registra o timeout, porém não retransmite.

Erros:

- se o servidor receber ids inválidos, a missão é marcada como concluída mesmo assim, mas com aviso no log;
- se o payload for menor que o esperado, a mensagem é ignorada.

2.6 `MSG_ACK` (Ambos os lados)

O **ACK** é utilizado para confirmar o recebimento confiável de três tipos de mensagem:

- `MSG_TELEMETRIA`
- `MSG_EQUIPE_DRONE`
- `MSG_CONCLUSAO`

O payload contém apenas um campo `status`, cujo valor indica qual evento está sendo confirmado:

- `ACK_TELEMETRIA` (0)
- `ACK_EQ_EQUIPE` (1)
- `ACK_CONCLUSAO` (2)

Comportamento no cliente:

- Um `ACK_TELEMETRIA` acorda a thread de telemetria, liberando o fluxo normal de envio.
- Um `ACK_EQ_EQUIPE` confirma o recebimento da ordem enviada pelo servidor.
- Um `ACK_CONCLUSAO` encerra o fluxo de conclusão da missão.

Comportamento no servidor:

- Um `ACK_EQ_EQUIPE` confirma que a equipe foi enviada e libera o servidor para novas ordens.
- Um `ACK_CONCLUSAO` confirma que o cliente recebeu a confirmação final.

Erros:

- Se o `status` for inválido, o ACK é ignorado.
- ACKs recebidos fora de contexto (por exemplo, sem telemetria enviada) geram logs informativos, mas não causam falha.

3 Estruturas de Dados

O sistema utiliza um conjunto de estruturas de dados projetadas para garantir organização, consistência e desempenho adequado para as operações exigidas. As estruturas são utilizadas tanto no servidor quanto no cliente, de maneira a permitir que ambos interpretem corretamente o grafo, as mensagens e o estado das missões.

A seguir descrevemos todas as estruturas utilizadas no trabalho, seu propósito, o momento em que são acessadas e as decisões de implementação associadas.

3.1 Representação do Grafo

O grafo da Amazônia Legal é carregado a partir do arquivo `grafo_amazonia_legal.txt`. Ele representa as cidades (vértices) e as distâncias rodoviárias entre elas (arestas). O grafo segue o modelo de lista de adjacências:

```
typedef struct adj {
    int id_vizinho;
    int peso;
    struct adj *prox;
} adj_t;

typedef struct {
    int id;
    char nome[100];
    int tipo;   /* 0 = regional, 1 = capital */
    adj_t *lista_adj;
} cidade_t;

typedef struct {
    int num_vertices;
    int num_arestas;
    cidade_t *cidades;
} grafo_t;
```

3.1.1 Uso no servidor

O servidor utiliza o grafo intensivamente:

- para executar o algoritmo de Dijkstra sempre que uma cidade entra em estado de alerta;
- para identificar quais cidades são capitais (equipes disponíveis);
- para determinar o nome e o tipo de cada cidade para fins de registro e logs.

3.1.2 Uso no cliente

O cliente utiliza o grafo para:

- conhecer o total de cidades a serem monitoradas;
- acessar os nomes das cidades ao imprimir logs das missões;
- validar índices recebidos em mensagens do servidor.

3.1.3 Tratamento de erros

- Caso o arquivo do grafo não exista ou seja inválido, o programa aborta com mensagem de erro, pois o sistema não pode funcionar sem este dado.
- Caso um `id_cidade` fora do intervalo seja lido em telemetria ou em mensagens do servidor, ele é simplesmente ignorado.

3.2 Estrutura de Alerta (Servidor)

Para gerenciar múltiplos alertas simultâneos, o servidor utiliza um vetor:

```
typedef struct {
    int ativo;
    int id_cidade;
    time_t timestamp;
    int equipe_id;
    int equipe_atuando;
} alerta_t;
```

Cada elemento do vetor representa um alerta ativo ou inativo.

3.2.1 Quando é usado

- criado quando o servidor recebe uma telemetria em que uma cidade está em estado de alerta;
- acessado ao receber um `ACK_EQ_EQUIPE`, marcando que a equipe está em missão;
- desativado ao receber `MSG_CONCLUSAO`.

3.2.2 Justificativa

Optou-se por um vetor fixo em vez de uma lista dinâmica:

- simplifica buscas (o número de cidades é pequeno);
- evita alocações e liberações constantes;
- diminui riscos de vazamento de memória.

3.3 Estados Globais das Equipes (Servidor)

O servidor mantém um vetor indicando se cada equipe está livre ou ocupada:

```
int *equipe_livre; /* 1 = livre, 0 = ocupada */
```

Cada **capital** corresponde a uma equipe.

3.3.1 Uso

- após executar Dijkstra, o servidor busca a capital livre com menor distância;
- ao enviar uma ordem de drone, a equipe permanece livre até que o `ACK_EQ_EQUIPE` chegue;
- ao receber `MSG_CONCLUSAO`, o servidor marca a equipe como livre.

3.3.2 Casos de erro

- caso não haja equipes livres, o servidor registra a impossibilidade de atender o alerta, mas continua executando normalmente.

3.4 Fila de Ordens Pendentes (Servidor)

O servidor mantém uma fila circular simples para ordens enviadas e que ainda esperam um `ACK_EQ_EQUIPE`:

```
int fila_pendentes[MAX_PENDENTES];
int pend_ini, pend_fim;
```

3.4.1 Uso

- sempre que uma ordem de drone é enviada, seu índice de alerta é colocado na fila;
- ao receber `ACK_EQ_EQUIPE`, retira-se o primeiro item da fila.

3.4.2 Tratamento de erros

- se a fila estiver cheia, o alerta não é enfileirado e o servidor registra o evento; neste caso, a equipe continuará livre até novo alerta.
- se um `ACK_EQ_EQUIPE` chega quando a fila está vazia, o servidor registra o erro mas mantém sua operação.

3.5 Status das Cidades (Cliente)

O cliente mantém o vetor:

```
int *status_cidade; /* 0 = normal, 1 = alerta */
```

Este vetor é atualizado exclusivamente pela *thread de monitoramento*.

3.5.1 Uso

- a thread 1 gera novos estados aleatórios;
- a thread 2 lê estes estados para construir a telemetria;
- a thread 3 utiliza este vetor apenas para validação de logs.

3.5.2 Concorrência

O acesso é protegido por um `mutex`, garantindo que nenhuma thread veja dados em estado inconsistente.

3.6 Estrutura da Missão Ativa (Cliente)

Para representar a missão atualmente em execução pelo drone, utiliza-se:

```
typedef struct {
    int ativa;
    int id_cidade;
    int id_equipe;
} missao_t;

missao_t missao_atual;
```

3.6.1 Uso

- preenchida pela thread 3 ao receber `MSG_EQUIPE_DRONE`;
- lida pela thread 4 para iniciar a simulação da missão;
- desativada pela thread 4 quando a missão termina.

3.6.2 Sincronização

- a thread 3 utiliza `pthread_cond_signal` para acordar a thread 4;
- a thread 4 espera em `pthread_cond_wait` até que haja missão ativa.

3.7 Sinalização por ACKs (Cliente)

Dois flags globais são utilizados para sincronizar recebimentos de ACK:

```
int ack_telemetria_ok;
int ack_conclusao_ok;
```

Cada um possui seu próprio par `mutex + condvar`.

3.7.1 Uso

- libera a thread 2 após um `ACK_TELEMETRIA`;
- libera a thread 4 após um `ACK_CONCLUSAO`.

3.7.2 Tratamento de erros

- se o ACK chega fora de contexto, nada quebra: a thread simplesmente ignora;
- timeouts são tratados como falhas não fatais.

4 Funcionamento do Servidor

O servidor é o núcleo lógico do sistema. Ele recebe as leituras de telemetria do cliente, identifica situações de alerta, executa o algoritmo de Dijkstra para selecionar a equipe mais adequada e coordena todas as etapas do despacho e conclusão de missões de drones. O servidor atua exclusivamente de forma reativa: ele só envia mensagens ao cliente como resposta direta a eventos de telemetria ou conclusão.

Nesta seção descrevemos detalhadamente o funcionamento do servidor, o fluxo de processamento de cada tipo de mensagem e o comportamento esperado em cada estágio, incluindo os mecanismos de detecção de erros e manuseio de condições inesperadas.

4.1 Inicialização

Ao iniciar, o servidor realiza as seguintes etapas, nesta exata ordem:

1. **Leitura do argumento `v4` ou `v6`**, que define o tipo de socket a ser criado. O programa aborta caso o argumento seja inválido.
2. **Criação do socket UDP**. Dependendo do modo escolhido:
   - em `v4`, utiliza-se a estrutura `sockaddr_in`;
   - em `v6`, utiliza-se `sockaddr_in6`.
3. **Bind** na porta definida no enunciado, permitindo que o servidor receba datagramas de qualquer cliente.
4. **Carregamento do grafo** a partir do arquivo `grafo_amazonia_legal.txt`.
5. **Identificação automática das capitais**, utilizadas como equipes.
6. **Inicialização do estado interno**:
   - vetor de equipes livres;
   - vetor de alertas ativos;
   - fila de pendências esperando `ACK_EQ_EQUIPE`.
7. Exibição de mensagem indicando que o servidor está pronto para receber telemetria.

Se qualquer uma dessas etapas falhar (por exemplo, arquivo do grafo inexistente ou socket inválido), o servidor aborta imediatamente, pois não pode operar de forma parcial.

4.2 Loop Principal de Execução

O servidor entra em um **loop infinito** no qual realiza:

- recepção de datagramas UDP;
- identificação do tipo da mensagem pelo cabeçalho;
- execução da rotina específica para cada tipo.

O servidor não possui threads; sua operação é estritamente sequencial, o que simplifica o gerenciamento de estados e elimina a necessidade de sincronização por mutex.

4.3 Processamento Geral de Pacotes

Para cada datagrama recebido, o servidor executa:

1. **Verificação do tamanho mínimo**. Pacotes menores que o cabeçalho são descartados.
2. **Leitura do tipo da mensagem** a partir de `header.tipo`.
3. **Validação do payload**. Se o tamanho informado no cabeçalho for maior que o número de bytes recebidos, o pacote é descartado por estar truncado.

4. **Despacho da mensagem** para a rotina correspondente:
    - `MSG_TELEMETRIA` → rotina de análise de sensores;
    - `MSG_CONCLUSAO` → rotina de finalização de missão;
    - `MSG_ACK` → rotina de confirmação;
    - `qualquer outro tipo` → registro e descarte.

Essa divisão garante modularidade e legibilidade do código.

4.4 Processamento da `MSG_TELEMETRIA`

Quando o servidor recebe uma mensagem de telemetria:

1. Valida-se o tamanho do payload.
2. Lê-se a quantidade de cidades enviadas.
3. Para cada cidade:
    - verifica se o ID está dentro do intervalo;
    - verifica se o estado é válido (0 ou 1);
    - em caso de dados inválidos, o servidor registra, mas não aborta.
4. As cidades com status `1` (alerta) são processadas individualmente.

4.4.1 Criação de novos alertas

Para cada cidade em alerta:

- Se já houver um alerta ativo associado àquela cidade, nada é feito.
- Caso contrário:
    1. cria-se um novo registro de alerta;
    2. executa-se o algoritmo de Dijkstra com origem na cidade em questão;
    3. identifica-se a capital mais próxima com equipe livre;
    4. se nenhuma estiver livre, o alerta permanece pendente.

4.4.2 Seleção da equipe

A equipe selecionada é sempre:

a capital com menor distância no grafo e cujo estado esteja "livre".

Se uma equipe for encontrada:

1. o servidor prepara o payload com o par *(id_cidade, id_equipe)*;
2. envia `MSG_EQUIPE_DRONE` ao cliente;
3. insere o alerta na fila de pendências aguardando ACK;
4. mantém a equipe livre até o recebimento do ACK.

4.4.3 Envio do ACK de telemetria

Após processar completamente a telemetria:

- o servidor monta um payload de `ACK_TELEMETRIA`;
- envia-o ao cliente;
- registra a confirmação no log.

4.4.4 Tratamento de erros da telemetria

- pacotes menores que o esperado são descartados;
- se mais cidades forem enviadas do que o total existente, o pacote é ignorado;
- se o cliente enviar telemetria em formato inconsistente, o servidor mantém operação normal e registra o evento.

4.5 Processamento da `MSG_EQUIPE_DRONE`

O servidor nunca deve receber esse tipo de mensagem. Se acontecer, ele:

1. registra o erro no log;
2. ignora completamente o datagrama.

4.6 Processamento da `MSG_CONCLUSAO`

Quando o servidor recebe uma conclusão de missão:

1. lê os campos `id_cidade` e `id_equipe`;
2. marca a equipe como livre;
3. desativa o alerta correspondente;
4. envia um `ACK_CONCLUSAO` ao cliente.

Tratamento de erros

- se o ID da equipe for inválido, o servidor registra e ignora;
- se o alerta não existir mais (caso raro), a mensagem é tratada como redundante;
- payloads incompletos são descartados.

4.7 Processamento da `MSG_ACK`

O servidor utiliza ACKs apenas para confirmar:

- recebimento de `MSG_EQUIPE_DRONE`;
- recebimento de `MSG_CONCLUSAO`.

Ao receber um ACK:

1. lê o campo `status`;
2. se o status for `ACK_EQ_EQUIPE`, remove o alerta correspondente da fila de pendências;
3. se o status for `ACK_CONCLUSAO`, apenas registra o evento.

Erros

- ACKs com status desconhecido são ignorados;
- se o ACK chegar quando não há pendências, o servidor registra o evento mas não falha.

4.8 Comportamento em IPv4 e IPv6

A implementação do servidor suporta ambos os modos. As diferenças são:

- **IPv4**: utiliza `AF_INET`, `sockaddr_in`, endereço `0.0.0.0`;
- **IPv6**: utiliza `AF_INET6`, `sockaddr_in6`, endereço `::`.

Todo o restante da lógica permanece idêntico, pois a manipulação do endereço remoto é abstraída pela função `recvfrom()`.

4.9 Robustez e comportamento em falhas

O servidor nunca encerra sua execução devido a erros de rede. As regras gerais são:

- pacotes inválidos são ignorados;
- tipos de mensagens desconhecidas são descartados;
- erros de leitura do grafo abortam o programa (inicialização é crítica);
- erros de socket são registrados, mas raramente fatais;
- a ausência de ACKs nunca impede novas missões.

O objetivo é manter o servidor sempre ativo e responsivo, independentemente do estado do cliente.

5 Funcionamento do Cliente

O cliente é responsável por simular o monitoramento ambiental e pela execução das missões de drones enviadas pelo servidor. O cliente utiliza **quatro threads concorrentes**, cada uma com responsabilidades definidas, e todas se comunicam por meio de variáveis compartilhadas protegidas por `mutex` e `condvar`.

O cliente também é capaz de operar tanto em IPv4 quanto em IPv6, de forma transparente, utilizando argumentos `v4` ou `v6` na linha de comando.

A seguir apresentamos o funcionamento completo do cliente, iniciando pela configuração inicial e avançando para o papel de cada thread.

5.1 Inicialização

Ao iniciar sua execução, o cliente realiza os seguintes passos:

1. **Interpretação do argumento `v4` ou `v6`**. Assim como o servidor, o cliente escolhe o tipo de socket a partir deste argumento.
2. **Criação do socket UDP**. Opcionalmente, define-se o endereço e porta do servidor usando a família de endereços correspondente.
3. **Carregamento do grafo**. O cliente utiliza o grafo exatamente como o servidor: para conhecer o total de cidades e seus nomes.
4. **Alocação do vetor de estados das cidades**. Cada cidade é marcada como `normal` no início.
5. **Inicialização de variáveis globais**, incluindo:
   - a estrutura `missao_atual`;
   - flags para indicar chegada de ACKs;
   - mutexes e condições de sincronização;
   - gerador de números aleatórios.
6. **Criação das quatro threads**:
   - **T1** — Thread de simulação de sensores;
   - **T2** — Thread de telemetria;
   - **T3** — Thread receptora (escuta de ordens);
   - **T4** — Thread de simulação de drone.

A partir deste ponto, todas as threads operam de forma concorrente e independente.

5.2 Thread 1 — Simulação dos Sensores

A Thread 1 (T1) simula continuamente o estado das cidades monitoradas. Ela opera com um intervalo fixo de aproximadamente 1—2 segundos. A cada iteração:

1. a thread adquire o `mutex` de estados;

2. percorre todas as cidades do grafo;
3. para cada cidade, gera um número aleatório;
4. se o número estiver na faixa dos 3% superiores, marca a cidade como **alerta**;
5. caso contrário, marca como **normal**;
6. libera o `mutex`.

5.2.1 Justificativa

Este comportamento reproduz exatamente as instruções que nos foram dadas, no qual cerca de 3% das cidades devem estar em alerta em cada ciclo de simulação. Optou-se por recalcular todos os estados a cada iteração para simplificar o modelo de sensores.

5.2.2 Erros e casos especiais

A thread T1 nunca bloqueia a execução do cliente:

- o acesso ao vetor de estados é sempre seguro via mutex;
- não há loops infinitos não controlados;
- nenhum erro de rede pode afetar a thread T1.

5.3 Thread 2 — Envio de Telemetria

A Thread 2 (T2) é responsável por enviar ao servidor, a cada 30 segundos, um pacote contendo o estado de todas as cidades. O fluxo é o seguinte:

1. A thread dorme por 30 segundos.
2. Ao acordar, adquire o mutex de estados e copia o vetor de `status_cidade`.
3. Monta o payload `MSG_TELEMETRIA`.
4. Envia a mensagem ao servidor via `sendto()`.
5. Aguarda o ACK correspondente.

5.3.1 Aguardando o ACK

Após enviar a telemetria, T2 aguarda um `ACK_TELEMETRIA`:

- é criada uma estrutura `timespec` com timeout de 5 segundos;
- T2 bloqueia-se em uma `condvar`;
- se o ACK chegar, T2 prossegue normalmente;
- se o timeout expirar, T2 tenta reenviar a telemetria mais uma vez;
- se as tentativas se esgotarem, T2 registra o erro e inicia novo ciclo de 30s.

5.3.2 Tratamento de erros

- erros no `sendto()` não interrompem o cliente, mas são logados;
- timeouts são esperados em UDP, e o cliente tenta retransmissão suave;
- se o servidor estiver offline, o cliente não trava — apenas registra a falha.

5.4 Thread 3 — Receptor de Mensagens

A Thread 3 (T3) tem papel central: ela escuta passivamente o socket UDP e reage a mensagens enviadas pelo servidor. A lógica é:

1. chama `recvfrom()` em loop infinito;
2. identifica o tipo da mensagem pelo cabeçalho;
3. despacha cada tipo para sua rotina correspondente.

5.4.1 Recebendo `MSG_EQUIPE_DRONE`

Quando T3 recebe uma ordem de drone:

1. lê `id_cidade` e `id_equipe`;
2. adquire o mutex da missão;
3. sobrescreve `missao_atual` com a nova missão;
4. define o campo `ativa = 1`;
5. libera o mutex;
6. envia um `ACK_EQ_EQUIPE` ao servidor;
7. sinaliza a Thread 4 para iniciar a simulação.

5.4.2 Recebendo `MSG_ACK`

Ao receber um ACK:

- se for `ACK_TELEMETRIA`, ativa `ack_telemetria_ok` e sinaliza T2;
- se for `ACK_CONCLUSAO`, ativa `ack_conclusao_ok` e sinaliza T4;
- se for `ACK_EQ_EQUIPE`, valida a ordem recebida anteriormente.

5.4.3 Tipos inválidos

Se T3 receber:

- uma mensagem desconhecida;
- payload menor do que o esperado;
- campo de status inválido;

então o cliente simplesmente registra o evento e descarta o pacote.

5.5 Thread 4 — Simulação das Missões de Drone

A Thread 4 (T4) representa o drone sendo enviado a campo. Ela opera da seguinte forma:

1. T4 se bloqueia inicialmente em uma `condvar`, aguardando que T3 informe uma nova missão.
2. Quando T3 sinaliza:
   (a) T4 copia a missão ativa;
   (b) dorme por um tempo aleatório entre 5 e 30 segundos, simulando o tempo de atuação;
   (c) prepara o `MSG_CONCLUSAO`;
   (d) envia a conclusão ao servidor;
   (e) aguarda o `ACK_CONCLUSAO` com timeout de 5 segundos.

5.5.1 Timeout do ACK de conclusão

Se o `ACK_CONCLUSAO` não chegar:

- T4 registra o evento no log;
- a missão é encerrada localmente de qualquer forma;
- nenhuma retransmissão é realizada.

5.5.2 Estados inválidos

Se T4 for sinalizada para iniciar uma missão enquanto outra ainda estiver em andamento, o comportamento previsto é:

- T4 encerra a missão anterior;
- substitui a missão por aquela recém-chegada;
- inicia a nova simulação.

Esta decisão segue a instrução explícita do enunciado de que o cliente deve sempre executar a missão mais recente.

5.6 Sincronização e Consistência

O cliente utiliza explicitamente:

- um mutex para os estados das cidades;
- um mutex para a missão ativa;
- um mutex para cada condvar de ACK.

Esses mecanismos garantem que:

- nenhuma thread leia estados inconsistentes;
- a missão ativa nunca esteja parcialmente escrita;
- ACKs não despertem threads incorretas.

5.7 Comportamento em IPv4 e IPv6

Assim como no servidor:

- `v4` cria um socket `AF_INET` e envia para um endereço IPv4;
- `v6` cria `AF_INET6` e envia para um endereço IPv6.

O código de envio e recepção é idêntico para ambas as famílias, sendo abstratizado pela função `sendto()` e pelo uso de ponteiros para `sockaddr`.

5.8 Robustez e Recuperação de Erros

O cliente foi projetado para ser resiliente a falhas:

- timeouts e perdas de pacotes são tratados como parte normal da operação;
- a telemetria é sempre reenviada de forma ordenada;
- falhas do servidor não derrubam o cliente;
- nenhuma thread depende de outra para manter-se viva;
- erros nos dados recebidos nunca travam a execução.

O sistema como um todo foi projetado para operar de forma contínua, mesmo sob condições de rede adversas ou instabilidade no servidor.

6 Compilação e Execução

Esta seção descreve detalhadamente o processo de compilação, estrutura dos binários gerados e instruções de execução tanto em IPv4 quanto em IPv6. Todas as instruções aqui apresentadas refletem exatamente o comportamento implementado no projeto.

6.1 Makefile

O projeto utiliza um `Makefile` para automatizar a compilação e organizar os binários em um diretório próprio. Ele contém regras para compilar todos os módulos (`server.c`, `client.c`, `grafo.c`) e gera dois executáveis:

- **`server`**
- **`client`**

O comando padrão:

```
make
```

executa a compilação completa. Caso seja necessário limpar todos os binários e objetos intermediários, utiliza-se:

```
make clean
```

6.2 Dependências e Organização dos Arquivos

Para compilar corretamente o projeto, os seguintes arquivos devem estar presentes no mesmo diretório que o `Makefile`:

- `server.c`
- `client.c`
- `grafo.c`, `grafo.h`
- `protocol.h`
- `grafo_amazonia_legal.txt`

O servidor e o cliente dependem diretamente dos módulos de grafo e do protocolo.

6.3 Execução dos Binários

Ambos os binários aceitam um único argumento obrigatório que define a família de endereços utilizada:

- `v4` → IPv4 (`AF_INET`)
- `v6` → IPv6 (`AF_INET6`)

6.3.1 Servidor

O servidor sempre deve ser iniciado antes dos clientes. A execução é feita por:

```
./server v4
```

ou

```
./server v6
```

Ao iniciar, o servidor:

- cria o socket adequado;
- realiza o *bind* na porta definida pela especificação;
- carrega o grafo;
- inicializa as estruturas de alerta e equipes;
- aguarda mensagens de telemetria.

O servidor imprime logs detalhados para cada:

- telemetria recebida;
- alerta detectado;
- seleção de equipe;
- ordem enviada ao cliente;
- missão concluída;
- ACK recebido.

6.3.2 Cliente

O cliente é executado como:

`./client v4`

ou

`./client v6`

Após iniciar, o cliente:

- cria o socket adequado;
- carrega o grafo;
- inicializa as threads de simulação, telemetria e controle de drones;
- inicia a transmissão periódica de telemetria a cada 30 segundos;
- aguarda ordens oriundas do servidor.

6.4 Fluxo de Execução Esperado

Uma execução típica segue a sequência:

1. O servidor é iniciado e aguarda telemetria.
2. O cliente é iniciado e começa a simular estados das cidades.
3. A cada 30 segundos, o cliente envia `MSG_TELEMETRIA`.
4. O servidor identifica alertas, executa Dijkstra e envia `MSG_EQUIPE_DRONE`.
5. O cliente inicia a Thread 4 para simular o deslocamento do drone.
6. Ao terminar a missão, o cliente envia `MSG_CONCLUSAO`.
7. O servidor responde com `ACK_CONCLUSAO`.
8. O ciclo recomeça.

6.5 Erros Comuns em Execução

O sistema foi projetado para lidar graciosamente com erros. Entre os mais comuns:

- **Porta ocupada**: ocorre se outro processo estiver utilizando a porta especificada. O servidor exibirá erro de `bind()` e não iniciará.
- **Arquivo do grafo não encontrado**: o servidor e o cliente abortam imediatamente, pois o grafo é essencial.
- **Tipo de mensagem desconhecida**: tanto o servidor quanto o cliente registram o evento e descartam o pacote.
- **Timeouts na espera por ACK**: o cliente registra o evento sem interromper sua execução.
- **Falha de conexão IPv6**: normalmente associada à ausência de suporte nativo na máquina executora; o programa recomenda o uso de `v4`.

6.6 Reprodutibilidade

Seguindo as etapas acima, qualquer pessoa pode reproduzir a execução completa do sistema, observando:

- telemetria sendo enviada;
- alertas detectados;
- drones sendo despachados;
- missões sendo concluídas;
- logs coerentes, ordenados e compatíveis com a especificação.

Essa seção garante que o sistema seja de fácil compilação e execução, atendendo diretamente às exigências formais.

7 Decisões de Implementação

Esta seção descreve e justifica as principais decisões de implementação adotadas no desenvolvimento do trabalho. O foco não é apenas informar como o sistema foi implementado, mas explicar *por que* cada solução técnica foi escolhida e de que maneira ela atende às exigências do enunciado.

7.1 Uso de UDP e ausência de conexões persistentes

Foi exigido explicitamente que toda a comunicação entre cliente e servidor ocorra por meio de datagramas UDP. Assim:

- Não há conexões persistentes.
- Não há handshake ou sessão.
- Não existe garantia de entrega, ordem ou deduplicação.

Por esse motivo, várias decisões foram tomadas em torno de robustez:

- O cliente implementa **timeouts** e **retransmissão** de telemetria.
- O servidor ignora mensagens incompletas ou truncadas.
- ACKs são utilizados para confirmar operações importantes.

Essa abordagem está diretamente alinhada com a filosofia do UDP: simplicidade, não confiabilidade, não conexão, tratamento explícito de falhas na aplicação.

7.2 Protocolo binário fixo

O `header_t` foi definido como uma estrutura fixa contendo:

- tipo (32 bits)
- tamanho do payload (32 bits)

Decisões implementadas:

1. O tipo sempre é interpretado antes do payload. Isso permite detectar e descartar pacotes desconhecidos imediatamente.
2. O tamanho do payload é validado contra o tamanho real recebido. Pacotes truncados são ignorados silenciosamente.
3. Os payloads usam estruturas fixas e bem definidas em `protocol.h`.

Alternativa rejeitada: *Uso de strings ou formato textual.* Justificativa para rejeição:

- maior custo de parsing,
- maior chance de inconsistências,
- não é alinhado ao modelo de comunicação em sistemas embarcados/telemetria.

7.3 Estrutura modular do código

O projeto foi dividido nos seguintes módulos:

- `grafo.c` / `grafo.h` — leitura do arquivo, adjacências e Dijkstra.
- `protocol.h` — definição de tipos de mensagens e estruturas binárias.
- `server.c` — lógica do servidor.
- `client.c` — lógica e threads do cliente.

Justificativas:

- O módulo do grafo pode ser testado isoladamente.
- O protocolo funciona como contrato entre cliente e servidor.
- `server.c` e `client.c` permanecem legíveis e organizados.

7.4 Implementação do grafo e do algoritmo de Dijkstra

O grafo é representado por listas de adjacência, e Dijkstra é implementado com complexidade $O(n^2)$. Essa decisão foi tomada com base em:

- o grafo contém poucas dezenas de vértices;
- a execução de Dijkstra não é feita continuamente, mas apenas quando há alerta;
- a complexidade é irrelevante para o tamanho do problema.

Alternativa rejeitada: *Uso de heap (fila de prioridade).* Rejeitada porque não traz benefício no contexto do trabalho e dificultaria desnecessariamente o código.

7.5 Escolha do método para simulação de sensores

A simulação segue exatamente a especificação: aproximadamente 3% das cidades devem estar em alerta a cada ciclo. Para isso:

- a cada iteração, cada cidade recebe um número aleatório;
- valores acima de 97% são interpretados como alerta.

Essa abordagem gera:

- distribuição simples e eficaz;
- independência entre ciclos;
- baixa complexidade computacional.

7.6 Motivação para uso de quatro threads no cliente

As quatro threads obedecem exatamente ao modelo do enunciado e impõem separação clara de responsabilidades:

T1 - Sensores: simula estados e atualiza variáveis compartilhadas.

T2 - Telemetria: envia dados a cada 30 segundos e aguarda ACK.

T3 - Receptora: única thread que faz `recvfrom()` no socket.

T4 - Drone: executa a missão e envia conclusão.

Essa organização minimiza riscos de deadlocks e facilita a sincronização:

- T3 acorda T2 quando chega `ACK_TELEMETRIA`.
- T3 acorda T4 quando chega uma ordem de drone.
- T4 aguarda `ACK_CONCLUSAO`.

O cliente permanece totalmente responsivo.

7.7 Uso de mutexes e condition variables

A sincronização foi feita com mutexes e `condvar`, seguindo boas práticas:

- **status das cidades** possui mutex próprio;
- **missão ativa** possui mutex próprio;
- **ACKs** possuem mutex e condvar independentes.

Isso evita que uma thread bloqueie indevidamente outras. Decisão importante:

> Usar condvars permite que T2 e T4 não fiquem em loop ativo aguardando ACKs, economizando CPU e mantendo o comportamento determinístico.

7.8 Tolerância a falhas e comportamento robusto

O sistema foi projetado para resistir a falhas comuns do UDP:

- mensagens duplicadas são tratadas como idempotentes;
- telemetria perdida é retransmitida;
- conclusões que se perdem não geram retransmissão;
- mensagens truncadas são descartadas imediatamente;
- tipos desconhecidos não geram falha — apenas logs.

7.9 Razões para não abortar em erros de tempo de execução

O cliente e o servidor foram projetados para:

- nunca abortarem por erros de rede;
- nunca encerrarem threads por erros de sincronização;
- sempre manterem as threads essenciais em loop infinito.

Essa decisão replica a filosofia de sistemas embarcados e distribuídos: *comunicação pode falhar, mas o sistema como um todo deve continuar rodando.*

7.10 Compatibilidade IPv4 e IPv6

A escolha por interpretar `v4` ou `v6` como argumento simplifica o uso do sistema e garante:

- compatibilidade máxima de teste;
- facilidade para o avaliador alternar modos;
- estrutura de código clara e modular.

A implementação também evita dependências externas e abstrai diferenças entre as famílias de endereços, usando ponteiros genéricos para `sockaddr`.

7.11 Log detalhado como parte do processo de correção

- todas as ações importantes geram logs;
- cada thread identifica suas operações com prefixos;
- cada mensagem enviada/recebida é descrita em texto humano;
- os logs permitem acompanhar facilmente o fluxo completo do sistema.

Essa decisão foi tomada para facilitar a verificação do sistema e refletir transparência de operação.

8 Conclusão

O desenvolvimento deste trabalho permitiu integrar, de forma coesa e funcional, diversos conceitos fundamentais de Redes de Computadores, incluindo comunicação por datagramas UDP, manipulação de sockets IPv4 e IPv6, concorrência com múltiplas threads e o uso de estruturas de dados adequadas para representar informações de topologia e executar algoritmos clássicos como o de Dijkstra.

A arquitetura final implementada demonstra a interação correta entre todos esses elementos: o cliente simula sensores distribuídos, gera telemetria periódica e reage dinamicamente às ordens enviadas pelo servidor; o servidor processa fluxos de dados potencialmente não confiáveis, detecta alertas em tempo real, executa cálculos de rotas e coordena as equipes de drones de forma precisa. Ambos os programas respeitam integralmente o protocolo binário definido, garantindo interpretabilidade, clareza e robustez na comunicação.

O sistema também incorpora mecanismos explícitos de tolerância a falhas, tais como timeouts, retransmissões, verificação de integridade dos pacotes e descarta de mensagens malformadas ou inesperadas. Essas decisões foram fundamentais para lidar com as limitações inerentes ao UDP e assegurar operação contínua mesmo sob condições adversas.

Do ponto de vista de concorrência, o cliente faz uso de quatro threads com responsabilidades claras e sincronizações cuidadosamente projetadas, evitando interferências indesejadas e garantindo determinismo na execução. A separação entre threads de monitoramento, telemetria, recepção e simulação de missões reproduz de forma fiel o comportamento de sistemas reais de monitoramento distribuído.

A modularização do código, aliada à elaboração de uma documentação detalhada e descritiva, facilita a reprodutibilidade da execução e a compreensão do raciocínio adotado em cada decisão de implementação. Cada módulo pode ser analisado, testado e executado de forma independente, reforçando a clareza estrutural do projeto.

Em síntese, o trabalho atinge todos os objetivos propostos: implementa um sistema distribuído completo, com cliente e servidor, operando tanto em IPv4 quanto em IPv6, utilizando telemetria, detecção de alertas, despacho de drones e confirmação de missões, tudo de forma robusta, modular e alinhada com princípios de redes de computadores e sistemas concorrentes.